Lenguajes Formales
Capítulo 2: Autómatas finitos
Sesión 1 de 7 — 2.1 Autómatas finitos deterministas

Notas de clase

Panorama del capítulo 2

De las palabras a las máquinas. En el capítulo 1 aprendimos a describir con precisión qué es una palabra y qué es un lenguaje (un conjunto de palabras). Ahora preguntamos algo distinto: ¿qué **procedimiento** puede decidir, para una palabra dada, si pertenece o no a un lenguaje concreto?

La idea central de este capítulo es la **memoria finita**: un procedimiento que, tras leer un prefijo de la entrada, solo conserva una entre un número *finito* de situaciones internas posibles.

Siete sesiones para el capítulo 2:

1. **Hoy:** Autómatas finitos deterministas (AFD)
2. Diseño y verificación de AFD mediante invariantes
3. Autómatas finitos no deterministas (AFN)
4. Transiciones ε
5. Determinización
6. Construcciones producto (intersección, unión, diferencia, complemento)
7. Problemas de decisión: pertenencia y vacuidad

Objetivos de la sesión de hoy

Al finalizar esta sesión, el estudiante podrá:

- definir formalmente un AFD $M = (Q, \Sigma, \delta, q_0, F)$ y explicar por qué δ debe ser una función **total**;
- representar un AFD mediante tabla y mediante diagrama de estados, reconociendo que describen el mismo objeto;
- definir la función de transición extendida $\widehat{\delta}$ y calcularla sobre palabras concretas;
- enunciar y usar la ley de concatenación para $\widehat{\delta}$;
- definir el lenguaje $L(M)$ reconocido por un AFD y decidir, para palabras concretas, si son aceptadas o rechazadas.

Observación

No se presupone ninguna teoría de autómatas previa. Sí se presupone el capítulo 1 completo (palabras, Σ^*, lenguajes, inducción sobre palabras).

Motivación: ¿cuánta memoria hace falta?

Pensemos en un procedimiento que lee una palabra símbolo a símbolo, de izquierda a derecha, **sin poder volver atrás** ni releer lo ya leído.

Para decidir, al final, si la palabra pertenece a un lenguaje L, el procedimiento solo puede apoyarse en lo que haya **recordado** del prefijo ya consumido.

Pregunta para la clase

Para reconocer $L_{\text{even}} = \{w \in \{a, b\}^* : \text{el número de } a \text{ en } w \text{ es par}\}$, ¿hace falta recordar *cuántas a* se han leído, o basta con recordar *algo mucho más simple*?

La respuesta —basta con recordar la **paridad**, un valor entre dos posibles— es el primer ejemplo de una idea muy general: a veces una cantidad finita de memoria es exactamente suficiente, aunque la entrada sea arbitrariamente larga.

Contenido

1. 2.1 Autómatas finitos deterministas

Estado: la abstracción de la memoria finita

La noción de **estado** abstrae toda la memoria interna disponible en un instante dado. En un autómata finito, el estado solo puede tomar uno entre un número **finito** de valores.

El estado actual debe contener **toda** la información que el procedimiento necesitará para reaccionar ante los símbolos que aún no ha leído: nada más se conserva.

Observación

Esto es una restricción fuerte: dos palabras distintas que dejan al autómata en el mismo estado serán, de ahí en adelante, **indistinguibles** para él.

Definición de AFD

Definición (Autómata finito determinista)

Un **autómata finito determinista** (AFD) es una quíntupla

$$M = (Q, \Sigma, \delta, q_0, F),$$

donde:

- Q es un conjunto **finito y no vacío** de **estados**;
- Σ es un alfabeto;
- $\delta\colon Q \times \Sigma \longrightarrow Q$ es la **función de transición**;
- $q_0 \in Q$ es el **estado inicial**;
- $F \subseteq Q$ es el conjunto de **estados finales**.

Interpretación operacional: si el autómata está en el estado q y lee el símbolo a, pasa al estado $\delta(q, a)$.

La palabra clave: *total*

Que $\delta\colon Q \times \Sigma \to Q$ sea una **función** exige dos condiciones simultáneas sobre cada pareja $(q, a) \in Q \times \Sigma$:

1. **Existencia:** $\delta(q, a)$ está definido para *toda* pareja (q, a) (totalidad de δ).
2. **Unicidad:** $\delta(q, a)$ es un *único* estado (no una colección de posibilidades).

Observación

Estas dos condiciones juntas son el sentido preciso de "determinista": desde cualquier estado, y para cualquier símbolo siguiente, el autómata **siempre** sabe a dónde ir y **nunca** duda entre varias opciones.

Una tabla que represente un AFD debe especificar un valor para **cada** elemento de $Q \times \Sigma$: ninguna casilla puede quedar vacía.

Tres maneras de describir la misma δ

La función δ es un único objeto matemático, pero puede **presentarse** de tres formas equivalentes:

- mediante una **fórmula** o regla explícita;
- mediante una **tabla** de doble entrada (estados $\times$ símbolos);
- mediante un **diagrama de estados** (un grafo dirigido con una flecha etiquetada por cada transición).

Observación

Cambiar de representación no cambia el AFD. En esta sesión usaremos las tres indistintamente, según convenga.

Ejemplo: paridad del número de a

Ejemplo (Diseño de un AFD para L_{even})

Sea $\Sigma = \{a, b\}$ y

$$L_{\text{even}} = \{w \in \{a, b\}^* : \text{el número de } a \text{ en } w \text{ es par}\}.$$

Tomamos $Q = \{q_{\text{e}}, q_{\text{o}}\}$, con la interpretación prevista:

$$q_{\text{e}} : \text{“número de } a \text{ leídas hasta ahora es par”}, \qquad q_{\text{o}} : \text{“... es impar”}.$$

Estado inicial q_{e} (cero a leídas: par). Estados finales $F = \{q_{\text{e}}\}$.

Observación

Por ahora esta interpretación solo **motiva** el diseño. Que sea correcta para **todas** las palabras se demostrará en la próxima sesión.

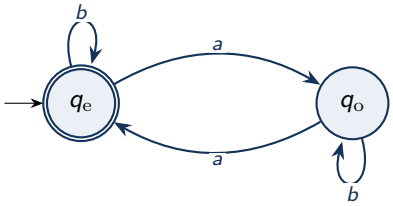
La transición de L_{even}, en tabla

$$\delta(q_{\text{e}}, a) = q_{\text{o}}, \qquad \delta(q_{\text{e}}, b) = q_{\text{e}}, \qquad \delta(q_{\text{o}}, a) = q_{\text{e}}, \qquad \delta(q_{\text{o}}, b) = q_{\text{o}}.$$

δ	a	b
q_{e}	q_{o}	q_{e}
q_{o}	q_{e}	q_{o}

Cada una de las cuatro casillas de $Q \times \{a, b\}$ tiene un valor: δ es total.

La transición de L_{even}, en diagrama



El doble círculo marca el estado final; la flecha entrante sin origen marca el estado inicial. Es exactamente la misma δ de la tabla anterior.

Contraejemplo: lo que no es un AFD

Contraejemplo (Una especificación parcial)

Sean $Q = \{q\}$, $\Sigma = \{a, b\}$, y supongamos que solo se declara $\delta(q, a) = q$, sin definir $\delta(q, b)$.

Esto **no** determina un AFD: δ no es una función sobre todo $Q \times \Sigma$, porque falta el valor en (q, b).

Error frecuente

Confundir un AFD con una "tabla con huecos". Si se quisiera estudiar un modelo con transiciones posiblemente indefinidas, habría que llamarlo explícitamente **AFD parcial** — un objeto distinto, que no usaremos en este curso: todos nuestros AFD son totales.

De letras a palabras: el problema

La función δ describe un único paso: recibe un estado y **una letra**. Pero para decidir si una palabra completa pertenece a $L(M)$ necesitamos saber en qué estado termina el autómata tras leer **toda** la palabra.

Como δ solo está definida sobre $Q \times \Sigma$ (un símbolo a la vez), necesitamos **extenderla** a $Q \times \Sigma^*$ (una palabra entera a la vez).

Función de transición extendida

Definición (Transición extendida)

Sea $M = (Q, \Sigma, \delta, q_0, F)$ un AFD. La **función de transición extendida** es

$$\widehat{\delta}\colon Q \times \Sigma^* \longrightarrow Q,$$

definida recursivamente por

$$\widehat{\delta}(q, \varepsilon) = q, \qquad \widehat{\delta}(q, wa) = \delta\big(\widehat{\delta}(q, w), a\big), \quad w \in \Sigma^*,\ a \in \Sigma.$$

Observación

δ y $\widehat{\delta}$ tienen **dominios distintos**: $\delta\colon Q \times \Sigma \to Q$ procesa una letra; $\widehat{\delta}\colon Q \times \Sigma^* \to Q$ procesa una palabra entera (incluida ε).

Por qué la definición tiene sentido

Toda palabra no vacía $x \in \Sigma^*$ se descompone de manera **única** como $x = wa$, con $w \in \Sigma^*$ y $a \in \Sigma$ la última letra de x (esto es una consecuencia directa de cómo se definió la concatenación en el capítulo 1). Por tanto:

- $\widehat{\delta}(q, x)$ queda determinado a partir del valor, **ya calculado**, en la palabra más corta w;
- el proceso siempre termina al llegar a ε, pues cada palabra tiene longitud finita.

En particular, si $a \in \Sigma$:

$$\widehat{\delta}(q, a) = \widehat{\delta}(q, \varepsilon a) = \delta\big(\widehat{\delta}(q, \varepsilon), a\big) = \delta(q, a).$$

La función extendida **coincide** con la original sobre palabras de longitud uno.

Error frecuente

La expresión $\delta(q, w)$ no está bien escrita cuando w no es un único símbolo: el segundo argumento de δ debe pertenecer a Σ, no a Σ^*. Para una palabra arbitraria se escribe siempre $\widehat{\delta}(q, w)$.

Ejemplo: calculando $\widehat{\delta}$ paso a paso

Ejemplo

Con el AFD de L_{even}, calculemos $\widehat{\delta}(q_{\text{e}}, aba)$ aplicando la definición de atrás hacia adelante:

$$\begin{aligned}
\widehat{\delta}(q_{\text{e}}, aba) &= \delta\big(\widehat{\delta}(q_{\text{e}}, ab), a\big) \\
&= \delta\Big(\delta\big(\widehat{\delta}(q_{\text{e}}, a), b\big), a\Big) \\
&= \delta\big(\delta(q_{\text{o}}, b), a\big) \\
&= \delta(q_{\text{o}}, a) \\
&= q_{\text{e}}.
\end{aligned}$$

De manera análoga, $\widehat{\delta}(q_{\text{e}}, aab) = q_{\text{e}}$ (verifíquenlo).

Ley de concatenación para $\widehat{\delta}$

Proposición (Ley de concatenación)

Sea $M = (Q, \Sigma, \delta, q_0, F)$ un AFD. Para cualesquiera $q \in Q$ y $u, v \in \Sigma^*$:

$$\boxed{\widehat{\delta}(q, uv) = \widehat{\delta}(\widehat{\delta}(q, u), v).}$$

En palabras: procesar uv de un tirón da el mismo resultado que procesar primero u, **olvidar** u y quedarse solo con el estado alcanzado, y continuar desde ahí con v.

Idea de la demostración

Se fija q, u y se induce sobre v (inducción sobre palabras, capítulo 1). Caso base $v = \varepsilon$: inmediato, pues $u\varepsilon = u$. Paso inductivo: usando $u(va) = (uv)a$ (asociatividad de la concatenación) y la cláusula recursiva de $\widehat{\delta}$ dos veces, ambos lados se reducen a $\delta(\widehat{\delta}(\widehat{\delta}(q, u), v), a)$.

Configuración implícita

Un AFD no tiene cinta de trabajo ni memoria auxiliar: tras leer un prefijo u de $w = uv$, **toda** la información interna disponible es el estado $\widehat{\delta}(q_0, u)$.

Podemos representar la situación, de manera puramente descriptiva, mediante el par $(\widehat{\delta}(q_0, u),\ v)$: estado alcanzado y sufijo pendiente. (Esto **no** introduce un modelo nuevo, solo nombra lo que ya sabemos.)

Observación

El prefijo u **ya consumido** no necesita conservarse literalmente: su único efecto relevante para el futuro está resumido en el estado $\widehat{\delta}(q_0, u)$. Esta es la ley de concatenación, leída operacionalmente.

Ejemplo: una configuración implícita

Ejemplo

Con el AFD de L_{even} y $w = aabb = a \cdot abb$: tras consumir $u = a$, $\widehat{\delta}(q_{\text{e}}, a) = q_{\text{o}}$, así que la situación es (q_{o}, abb). Continuando desde q_{o}: $\widehat{\delta}(q_{\text{o}}, abb) = q_{\text{e}}$.

Por la ley de concatenación,

$$\widehat{\delta}(q_{\text{e}}, aabb) = \widehat{\delta}(\widehat{\delta}(q_{\text{e}}, a), abb) = \widehat{\delta}(q_{\text{o}}, abb) = q_{\text{e}},$$

que coincide con procesar $aabb$ de principio a fin:

$$q_{\text{e}} \xrightarrow{a} q_{\text{o}} \xrightarrow{a} q_{\text{e}} \xrightarrow{b} q_{\text{e}} \xrightarrow{b} q_{\text{e}}.$$

Lenguaje reconocido por un AFD

Definición (Lenguaje reconocido)

Sea $M = (Q, \Sigma, \delta, q_0, F)$ un AFD. El **lenguaje reconocido por** M es

$$L(M) = \{w \in \Sigma^* : \widehat{\delta}(q_0, w) \in F\}.$$

Una palabra w es **aceptada** por M si $\widehat{\delta}(q_0, w) \in F$, y **rechazada** en caso contrario.

La aceptación se decide **solo** al final, tras leer la palabra completa.

Caso particular inmediato:

$$\varepsilon \in L(M) \iff \widehat{\delta}(q_0, \varepsilon) \in F \iff q_0 \in F.$$

Ejemplo: aceptación y rechazo

Ejemplo

En el AFD de L_{even} ($F = \{q_{\text{e}}\}$):

$$\varepsilon, \quad b, \quad aa, \quad aba \quad \text{son } \textbf{aceptadas} \qquad (\widehat{\delta}(q_{\text{e}}, \cdot) = q_{\text{e}});$$

$$a, \quad ab, \quad aaa \quad \text{son } \textbf{rechazadas} \qquad (\widehat{\delta}(q_{\text{e}}, \cdot) = q_{\text{o}}).$$

Error frecuente

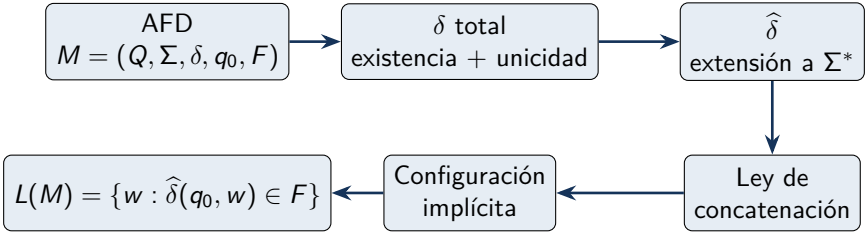
aaa visita el estado final q_{e} tras el prefijo *aa*, pero la última *a* produce $q_{\text{e}} \xrightarrow{a} q_{\text{o}}$: la palabra completa termina en $q_{\text{o}} \notin F$ y es **rechazada**. Visitar un estado final **a mitad de camino** no implica aceptación: solo cuenta el estado alcanzado **al final**.

Actividad rápida (10 minutos)

Trabajen en parejas. Sea $\Sigma = \{a, b\}$ y consideren $L_{\text{termina-b}} = \{w \in \{a, b\}^* : w \text{ termina en } b\}$ (incluyendo convencionalmente que ε **no** termina en b).

1. Propongan Q, q_0, F y δ (tabla o diagrama) para un AFD que reconozca $L_{\text{termina-b}}$.
2. Calculen $\widehat{\delta}(q_0, abba)$ y $\widehat{\delta}(q_0, bab)$ paso a paso.
3. ¿$\varepsilon \in L(M)$ con su diseño? Verifíquenlo con la fórmula $\varepsilon \in L(M) \iff q_0 \in F$.
4. **Verdadero o falso:** si un AFD visita un estado final en algún momento del procesamiento, la palabra completa es aceptada.

Síntesis de la sesión



Lo que viene: en la sesión 2 demostraremos, de manera sistemática y sin casos particulares, que un AFD diseñado como el de L_{even} realmente reconoce el lenguaje que se pretendía.

Errores frecuentes de hoy

- Tratar δ como si pudiera quedar indefinida en alguna pareja (q, a): en este curso, todo AFD es total.
- Escribir $\delta(q, w)$ con w de longitud distinta de uno: para palabras se usa $\widehat{\delta}$.
- Pensar que visitar un estado final en algún punto del procesamiento basta para aceptar: solo cuenta el estado **final del recorrido completo**.
- Olvidar que $\varepsilon \in L(M)$ depende únicamente de si $q_0 \in F$.

Trabajo autónomo antes de la próxima sesión

- Repasar estas diapositivas: rehacer, sin mirar, la definición de AFD, la de $\widehat{\delta}$ y la de $L(M)$, con un ejemplo propio (otro AFD de dos o tres estados).
- Resolver, del banco de ejercicios del capítulo 2:
  - Diseñar un AFD para un lenguaje concreto (p. ej. "w contiene al menos dos a" o "$|w|$ es múltiplo de 3") y calcular $\widehat{\delta}$ sobre varias palabras.
  - Demostrar, por inducción sobre palabras, que para el AFD de L_{even} se cumple $\widehat{\delta}(q_{\text{e}}, w) = q_{\text{e}}$ si y solo si w tiene un número par de a (adelanto informal de lo que formalizaremos mañana).
- Pensar: ¿por qué la ley de concatenación garantiza que *no importa* en qué punto se corte una palabra para procesarla por partes?

Lecturas recomendadas

- J. E. Hopcroft, R. Motwani, J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation.*
- D. C. Kozen, *Automata and Computability.*
- M. Sipser, *Introduction to the Theory of Computation.*
- P. Linz, *An Introduction to Formal Languages and Automata.*
- E. Rich, *Automata, Computability and Complexity.*

Próxima sesión: **2.2 Diseño y verificación de AFD**